

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	S.Blessy	Reg.No.:	192421053
Department:	Btech.IT	Date:	01.08.26

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry lookahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bitpair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I _____Blessy.S(192421053)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Blessy.s

Signature of the student:**MARKS ALLOCATION**

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation

Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided
---------------------------	-----	--	---	--	---

Assessment Tool 2 — Debugging & Optimisation Task (CO2)

Course Code: CSA12 | Course Title: Computer Architecture

NAME: Blessy.S

REG NO: 192421053

Question 1

Debug incorrect signed addition of +120 and +50 in 8-bit 2's complement; identify overflow and propose detection/handling.

1. Identification of the Bug

Adding +120 and +50 gives a mathematically correct sum of 170. In an 8-bit signed (2's complement) register the representable range is only -128 to +127, so 170 cannot be represented. This is a classic signed overflow bug.

2. Step-by-Step Binary Analysis

8-bit 2's complement representations:

+120 = 0 111 1000

+50 = 0 011 0010

Bit-by-bit addition (LSB → MSB), tracking the carry into and out of each bit:

Bit position	7 (sign)	6	5	4	3	2	1	0
Operand A (120)	0	1	1	1	1	0	0	0
Operand B (50)	0	0	1	1	0	0	1	0
Carry-in	1	1	1	1	0	0	0	0
Result bit	1	0	1	0	1	0	1	0

Result bit pattern = 1010 1010, which is the 2's complement encoding of -86 (not +170). The carry generated into the sign bit (bit 7) is 1, while the carry generated out of the sign bit is 0.

3. Root Cause

- Two positive operands were added, but the result's sign bit flipped to 1 (negative) — a mathematically impossible outcome for two positive inputs.
- This happens because the true sum (170) exceeds the maximum positive value representable in 8 bits (127), causing it to wrap around into the negative range.
- The bug in the program is that it trusts the raw ALU result without checking the overflow condition, so it silently accepts -86 as the answer.

4. Overflow Detection — the Fix

Overflow can be detected using either of two equivalent hardware rules:

- Carry-based rule: $V = \text{Cin} \oplus \text{Cout}$ at the sign bit (bit 7). Here $\text{Cin} = 1$ and $\text{Cout} = 0$, so $V = 1 \rightarrow$ overflow confirmed.
- Sign-based rule: overflow occurs when both operands have the same sign but the result has the opposite sign. Here both operands are positive (sign bit 0) but the result is negative (sign bit 1) $\rightarrow$ overflow confirmed.

5. Proposed Solution

- Hardware: implement/enable the overflow flag (V) in the ALU status register using the $\text{Cin} \oplus \text{Cout}$ rule at the MSB, and raise a trap/interrupt when $V = 1$ for critical arithmetic.
- Software: before or after the add, explicitly check — if $(\text{signA} == \text{signB}) \ \&\& \ (\text{signResult} != \text{signA}) \rightarrow$ overflow — and handle it (saturate to $+127/-128$, throw an exception, or promote to a wider type).
- Optimization: for applications where overflow is likely, widen the datapath (use 16-bit instead of 8-bit) or use saturating arithmetic instructions available in many embedded/DSP instruction sets, avoiding silent wraparound.

6. Verification

Repeating the same addition in 16-bit: $0000\ 0000\ 0111\ 1000$ (120) + $0000\ 0000\ 0011\ 0010$ (50) = $0000\ 0000\ 1010\ 1010$ = 170 (correct, no overflow since $170 < 32767$). This confirms the earlier 8-bit result was invalid due to range overflow, not a logic error in the adder itself.

```
a = 120
b = 50

result = (a + b) & 0xFF

# Convert to signed 8-bit
if result >= 128:
    result -= 256

print("120 + 50 =", result)

if a + b > 127 or a + b < -128:
    print("ERROR: Overflow occurred!")
else:
    print("No Overflow")
```

```
>>> ===== RESTART: C:/Users/Admin/Documents/ss.py =====
120 + 50 = -86
ERROR: Overflow occurred!
>>>
```

Question 2

Debug delay bottleneck in a ripple carry adder for a real-time system and propose a carry look-ahead adder (CLA) solution.

1. Identification of the Bug/Bottleneck

A ripple carry adder (RCA) is built from n full adders (FAs) connected in a chain, where each FA's carry-out feeds the carry-in of the next stage. The performance problem is not a logic error but a structural/timing bottleneck: the final sum and carry-out are only valid after the carry has rippled through all n stages.

2. Debugging the Carry Propagation Path

- Stage 0 must compute its carry before Stage 1 can begin its own carry computation; Stage 1 before Stage 2, and so on — a strictly sequential dependency chain.
- Worst-case critical path delay $T_d \approx n \times t_{\text{carry}}$, where t_{carry} is the delay of one full adder's carry stage. For $n = 32$ or 64 bits, this delay dominates the clock period.
- In a real-time system, this means the ALU cannot guarantee its result within the required cycle time, causing setup-time/timing violations or forcing a lower clock frequency — unacceptable for real-time deadlines.

3. Root Cause

The delay grows linearly, $O(n)$, with the number of bits because every bit must literally wait for the physical carry signal from the previous bit — there is no parallelism in how the carry is computed.

4. Optimized Solution: Carry Look-Ahead Adder (CLA)

A CLA removes the sequential dependency by computing all carries in advance from the input bits directly, using two auxiliary signals per bit:

Generate: $G_i = A_i \cdot B_i$ (this stage generates a carry on its own)

Propagate: $P_i = A_i \oplus B_i$ (this stage passes an incoming carry through)

Every carry can then be expressed purely in terms of G_i , P_i and the initial carry-in C_0 , without waiting for the previous FA to finish:

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

$$C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

Because each C_i is a two-level AND-OR (sum-of-products) function of the primary inputs, all carries are available after a fixed, small gate delay (≈ 2 gate delays), independent of n , instead of rippling stage by stage.

5. How Delay Is Reduced

- Ripple carry adder: delay grows linearly, $O(n)$, with word length.
- Carry look-ahead adder: delay is effectively $O(1)$ for a single-level CLA block (typically 4 bits) and $O(\log n)$ when CLA blocks are combined hierarchically (block-carry-lookahead) for wider words, since carries between blocks are also look-ahead generated instead of rippled.
- Trade-off: CLA logic grows more complex (more gates, higher fan-in) as n increases, so practical designs use a hierarchy of small CLA blocks (e.g., 4-bit CLA units combined with a second level of look-ahead) to balance speed against gate count and fan-in limits.

6. Conclusion

For the real-time system, replacing the ripple carry adder with a hierarchical (multi-level) carry look-ahead adder removes the linear carry-propagation bottleneck, meeting the required timing budget while keeping area overhead manageable.

```
a = 15
b = 1

# Ripple Carry: carry moves bit by bit
ripple = a + b
print("RCA Output:", ripple)
print("RCA: High delay due to carry propagation")

# Carry Lookahead: carry calculated in parallel
cla = a + b
print("CLA Output:", cla)
print("CLA: Low delay using parallel carry calculation")
```

```
>>>
===== RESTART: C:/Users/Admin/Documents/ss.py =====
RCA Output: 16
RCA: High delay due to carry propagation
CLA Output: 16
CLA: Low delay using parallel carry calculation
~::~
```

Question 3

Debug Booth's algorithm giving incorrect results for certain negative operands; analyze bit-pair checking, shifting and sign extension.

1. Recap of Booth's Algorithm Rule

Booth's algorithm examines the current multiplier bit Q_0 together with the previous bit Q_{-1} (initialized to 0) and performs one of the following on the AC (accumulator) register, followed by an arithmetic right shift of the combined AC- Q_{-1} register:

Q ₀ Q ₋₁	Operation	Then
0 0	No arithmetic operation	Arithmetic shift right (ASR)
0 1	AC = AC + M (add multiplicand)	ASR
1 0	AC = AC – M (subtract multiplicand)	ASR
1 1	No arithmetic operation	ASR

2. Where the Errors Typically Creep In

- Logical shift instead of arithmetic shift: using a logical right shift discards the sign bit information, corrupting negative partial products. The shift must be an arithmetic shift right (ASR) that replicates the MSB (sign bit) of AC into the vacated position.
- Incorrect 2's complement of M: the '10' case requires $AC = AC + (2\text{'s complement of } M)$. If $-M$ is computed incorrectly (e.g., only inverted, without adding 1), every subtraction step produces a wrong partial result.
- Wrong iteration count: the algorithm must run exactly n times for an n -bit multiplier (with $Q-1$ initialized to 0 before the first iteration). Running one iteration too few or too many misaligns the final product.
- Sign extension omitted for the multiplicand: when M itself is negative, it must be stored in full 2's complement form in the AC-width register before the add/subtract step, otherwise the upper bits are not correctly sign-extended and the arithmetic is wrong.
- Edge case — most negative operand: a value such as -128 in an 8-bit field has no positive counterpart, so its magnitude cannot be simply negated; this boundary case needs a widened register (guard bit) or special handling.

3. Debugging Walkthrough (Example)

Multiplicand $M = -3$ (1101 in 4-bit), Multiplier $Q = 3$ (0011), $n = 4$ bits.

Init: $AC = 0000$ $Q = 0011$ $Q-1 = 0$ $M = 1101$ $-M = 0011$

Step1: $Q0Q-1=1,0 \rightarrow AC=AC-M=AC+(-M)=0000+0011=0011 \rightarrow ASR \rightarrow AC \ Q \ Q-1 = 0001 \ 1001 \ 1$

Step2: $Q0Q-1=1,1 \rightarrow \text{no-op} \rightarrow ASR \rightarrow 0000 \ 1100 \ 1$

Step3: $Q0Q-1=0,1 \rightarrow AC=AC+M=0000+1101=1101 \rightarrow ASR \rightarrow 1110 \ 0110 \ 0$

Step4: $Q0Q-1=0,0 \rightarrow \text{no-op} \rightarrow ASR \text{ (sign-extend)} \rightarrow 1111 \ 0011 \ 0$

Final $AC-Q = 1111 \ 0011 = -9$, which correctly equals -3×3 . If a logical shift had been used in Step 4 instead of an arithmetic (sign-extending) shift, the leading bit would incorrectly become 0 instead of 1, giving a wrong final product — this is the exact class of bug described in the problem.

4. Corrections to Apply

- Always use arithmetic (sign-extending) right shifts across the $AC-Q-Q-1$ register, never a logical shift.
- Compute $-M$ once, correctly, using full 2's complement (invert all bits and add 1) at the required register width.
- Initialize $Q-1 = 0$ and run for exactly n iterations for an n -bit operand.
- Sign-extend the multiplicand M into the AC width before any add/subtract so negative multiplicands are handled correctly.
- Guard against the most-negative-number edge case by using one extra bit of headroom or special-casing it.


```
def booth(a, b):
    result = a * b
    print("A =", a, " B =", b)
    print("Result =", result)

    if a < 0 or b < 0:
        print("Signed multiplication using 2's complement")
        print("Check: bit-pair, arithmetic shift & sign extension")
    return result

booth(-5, 3)|
```

```
===== RESTART: C:/Users/Admin/Documents/ss.py =====
A = -5  B = 3
Result = -15
Signed multiplication using 2's complement
Check: bit-pair, arithmetic shift & sign extension
|
```

Question 4

Debug inefficiency in restoring division (repeated restoration) in an embedded system and propose non-restoring division.

1. Recap of Restoring Division

For each of the n bits: shift the Remainder-Quotient (A-Q) register left by 1, subtract the divisor M from the upper half ($A = A - M$). If the result is negative, the quotient bit is set to 0 and the divisor is added back (restored) to A ; if non-negative, the quotient bit is set to 1 and no restoration is needed.

2. Identification of the Inefficiency

- Every time a trial subtraction goes negative, the algorithm performs a second operation (the restore/add-back) purely to undo the previous subtraction — this restore step contributes nothing to the final result except recovering the correct remainder for the next iteration.
- In the worst case (quotient bits mostly 0), nearly every iteration needs both a subtract and a restore, i.e., up to $2n$ arithmetic operations for an n -bit division instead of the theoretical minimum of about n operations.
- In an embedded, real-time or power-constrained system, these redundant add-back operations directly increase execution cycles, latency and energy consumption — exactly the inefficiency described in the scenario.

3. Root Cause

The inefficiency is structural: restoring division always speculatively subtracts and then unconditionally checks/corrects, rather than using the sign of the previous remainder to decide the next operation in advance.

4. Optimized Solution: Non-Restoring Division

Non-restoring division removes the separate restore step by using the sign of the current remainder to choose the next operation directly:

- If the previous remainder A is non-negative: shift A-Q left, then subtract M ($A = A - M$); set quotient bit 1 if result ≥ 0 , else 0.
- If the previous remainder A is negative: shift A-Q left, then add M ($A = A + M$) instead of subtracting; set quotient bit 1 if result ≥ 0 , else 0.
- No add-back/restore happens inside the loop at all — only if the very final remainder is negative is a single corrective addition of M performed at the end (one restore total, not one per iteration).

5. How This Improves Performance

- Operation count drops from up to $2n$ (restoring) to exactly $n+1$ (non-restoring) add/subtract operations for an n -bit division — roughly a 2x reduction in the worst case.
- Fewer operations directly means fewer clock cycles per division and lower dynamic power consumption, both critical for embedded/real-time constraints.
- The control logic is also simpler in one sense: the decision of add vs. subtract is made purely from the sign bit of the previous remainder, with no separate restoration micro-step to sequence.

6. Conclusion

Replacing restoring division with non-restoring division eliminates the redundant add-back operations identified as the bottleneck, cutting worst-case arithmetic operations roughly in half and making the divider suitable for tight embedded real-time budgets.

```
File Edit Format Run Options Window Help
```

```
dividend = 20
```

```
divisor = 3
```

```
# Restoring Division
```

```
q1 = dividend // divisor
```

```
r1 = dividend % divisor
```

```
print("Restoring: Q =", q1, "R =", r1)
```

```
print("Problem: Repeated restoration increases delay")
```

```
# Non-Restoring Division
```

```
q2 = dividend // divisor
```

```
r2 = dividend % divisor
```

```
print("Non-Restoring: Q =", q2, "R =", r2)
```

```
print("Optimized: Avoids repeated restoration")
```

```
===== RESTART: C:/Users/Admin/Documents/ss.py =====
```

```
Restoring: Q = 6 R = 2
```

```
Problem: Repeated restoration increases delay
```

```
Non-Restoring: Q = 6 R = 2
```

```
Optimized: Avoids repeated restoration
```

Question 5

Debug inaccurate IEEE 754 single-precision addition of a very small and a very large number; analyze exponent alignment, normalization and rounding; propose optimizations.

1. IEEE 754 Single Precision Format

1 sign bit + 8 exponent bits (bias 127) + 23 explicit mantissa bits (24 bits of precision including the implicit leading 1), giving roughly 7 decimal digits of precision.

2. Debugging the Addition Process

- Exponent alignment: to add two floating-point numbers, the ALU aligns them to the same (larger) exponent by shifting the mantissa of the smaller-magnitude number right by the exponent difference.
- Precision loss / “swamping”: if the exponent difference exceeds about 24, the smaller number's mantissa is shifted completely out of the 24-bit mantissa field. Even with guard, round and sticky bits (typically 3 extra bits in hardware), a sufficiently large gap makes the small operand's contribution vanish entirely — the sum simply equals the larger number.
- Normalization: after the addition, the result may need re-normalizing (adjusting the mantissa and exponent so the leading bit is 1). Skipping or mishandling this step after the aligned add is a second common source of error.
- Rounding: the default IEEE 754 round-to-nearest-even mode can introduce additional error at each step; when this happens repeatedly across many additions (e.g., summing a long series), rounding errors accumulate (“catastrophic cancellation/accumulation”).

3. Root Cause

The inaccuracy is not a hardware fault but an inherent precision limitation: single precision only carries 24 bits of mantissa, so any pair of numbers whose magnitudes differ by more than roughly 2^{24} ($\approx 1.6 \times 10^7$ times) cannot both be represented exactly in the same aligned sum — the smaller value is effectively lost.

4. Illustrative Example

Large: 1.0×10^8 (needs ~ 27 bits to represent exactly alongside small value)

Small: 1.0×10^0

Ratio = 10^8 , which exceeds the $\sim 1.6 \times 10^7$ threshold that 24 mantissa bits can resolve, so after exponent alignment the small value's bits fall entirely outside the mantissa window and the computed sum rounds back to 1.0×10^8 , silently dropping the small term.

5. Optimization Techniques

- Use double precision (64-bit: 11-bit exponent, 52-bit mantissa) when the application needs to combine values of very different magnitudes — this raises the “swamping” threshold from $\sim 2^{24}$ to $\sim 2^{53}$, covering far more real-world cases.
- Kahan (compensated) summation: maintain a separate running “compensation” variable that captures the low-order bits lost on each addition and feeds them back into subsequent sums, dramatically improving accuracy without changing the underlying precision.
- Reorder/accumulate small values first: sum numbers in increasing order of magnitude, or use pairwise (tree) summation, so that operands being added are closer in magnitude at each step, reducing the exponent gap and precision loss.

- Use extended-precision or fused multiply-add (FMA) accumulators internally where available, reducing the number of separate rounding steps.
- Algorithmically rescale the data (e.g., subtract a common large offset before summation) so that the values being added are of comparable magnitude, then add the offset back at the end.

6. Conclusion

The inaccuracy stems from the fundamental exponent-alignment/mantissa-truncation limitation of IEEE 754 single precision, not a bug in the adder logic. Debugging confirms the issue is precision-related (verified by checking the exponent difference against the ~24-bit mantissa threshold), and the practical fix is a combination of switching to double precision and applying an accuracy-preserving summation algorithm such as Kahan summation.

```
import numpy as np

a = np.float32(1e20)
b = np.float32(1e-5)

result = a + b

print("A =", a)
print("B =", b)
print("Result =", result)

if result == a:
    print("ERROR: Small value lost due to exponent alignment/rounding")
    print("Solution: Use double precision")
```

```
===== RESTART: C:/Users/Admin/Documents/ss.py =====
Restoring: Q = 6 R = 2
Problem: Repeated restoration increases delay
Non-Restoring: Q = 6 R = 2
Optimized: Avoids repeated restoration
|
```